ESCOLA POLITÉCNICA DA UNIVERSIDADE DE SÃO PAULO

DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E SISTEMAS DIGITAIS

EXPERIÊNCIA 4: TRENA DIGITAL COM SAÍDA SERIAL

Relato da Experiência
Bancada A5 – Turma 1 – Prof. Edson Midorikawa

Gabriel Zocal Santos – NUSP 15454039
Caio Nassar de Souza – NUSP 15444601
Victor Bertelli – NUSP 15490941
Erwan Pierre Didelot – NUSP 18403997

PCS3645 -- LABORATÓRIO DE PROJETO DE SISTEMAS DIGITAIS II

22 de setembro de 2026

Sumário

1 Atividade 1: Projeto e Simulação do Circuito da Trena Digital **3**
1.1 Especificação . . . 3
1.2 Arquitetura . . . 4
1.3 Funcionamento . . . 7
1.4 Componentes reutilizados e alterações . . . 8
1.5 Decisões de projeto e limitações . . . 12
1.6 Plano de testes . . . 13
1.7 Resultados da simulação . . . 16
1.8 Plano de execução experimental . . . 22

2 Atividade 2: Preparação do Projeto para Síntese na Placa FPGA **25**
2.1 Designação de pinos . . . 25
2.2 Sinais de depuração adicionais . . . 26
2.3 Tratamento dos sinais ativos em nível baixo . . . 26
2.4 Restrições de tempo e resultado da compilação . . . 27
2.5 Arquivos entregues . . . 27

3 Atividade 4: Desafio – Modo de Medida Dupla **28**
3.1 Especificação . . . 28
3.2 Projeto . . . 28
3.3 Testes de simulação . . . 30
3.4 Resultados da simulação . . . 30
3.5 Validação . . . 32

1 Atividade 1: Projeto e Simulação do Circuito da Trena Digital

1.1 Especificação

A Trena Digital mede a distância até um objeto com o sensor ultrassônico HC-SR04 e apresenta o resultado, em centímetros, de duas formas: em três displays de 7 segmentos (formato BCD) e por comunicação serial assíncrona 115200/7E1 (115200 bauds, 7 bits de dados, paridade par e 1 *stop bit*). A Tabela 1 resume a interface do módulo principal `exp4_trena`.

Tabela 1: Interface do módulo `exp4_trena`.

Sinal	Direção	Bits	Descrição
`clock`	Entrada	1	Clock de 50 MHz da placa (`CLOCK_50`).
`reset`	Entrada	1	Reset assíncrono, ativo em nível alto. Leva o circuito à condição inicial.
`mensurar`	Entrada	1	Dispara uma medida na borda de subida. Vem do botão `KEY0`, ativo em nível baixo.
`echo`	Entrada	1	Pulso do HC-SR04 com largura proporcional à distância.
`trigger`	Saída	1	Pulso de 10 μs que dispara o HC-SR04.
`saida_serial`	Saída	1	Saída serial 115200/7E1.
`medida0/1/2`	Saída	7	Unidade, dezena e centena, já codificadas para os displays.
`pronto`	Saída	1	Pulso de 1 ciclo ao fim de cada operação (medida e envio).
`db_mensurar`, `db_echo`, `db_trigger`, `db_saida_serial`	Saída	1	Cópias de sinais para depuração.
`db_estado`	Saída	7	Estado da UC da trena (HEX5).
`db_estado_sensor`, `db_estado_tx`	Saída	7	Estados das UCs do sensor e do transmissor (HEX4 e HEX3).
`db_tick`	Saída	1	Tick de *baud rate* do transmissor (LEDR2).

Os requisitos a seguir foram extraídos do roteiro e são usados para relacionar cada caso de teste ao que ele verifica:

R1 O reset leva o circuito à condição inicial, com `medida` igual a 000.

R2 Uma medida é disparada somente na borda de subida de `mensurar`. Manter o sinal em '1' não dispara novas medidas.

R3 A distância é apresentada em centímetros, em BCD, em três displays (`medida0` = unidade, `medida1` = dezena, `medida2` = centena).

R4 O valor medido permanece nos displays até o próximo acionamento de `mensurar` ou até o reset.

R5 A saída `pronto` é ligada quando a medida é atualizada.

R6 A medida é enviada pela porta serial como três caracteres ASCII (centena, dezena e unidade) seguidos de `#` (23H), em 115200/7E1. Exemplo: 172 cm → 31H, 37H, 32H, 23H.

R7 O projeto é síncrono e dividido em fluxo de dados (FD) e unidade de controle (UC), e a UC é especificada por um diagrama ASM. Os circuitos das experiências anteriores são reutilizados sem elementos de interface externa.

R8 Todos os sinais do projeto são ativos em nível alto, e a inversão do botão `KEY0` é tratada no código.

1.2 Arquitetura

O módulo `exp4_trena` interliga quatro partes: o detector de borda de `mensurar`, a UC (`exp4_trena_uc`), o FD (`exp4_trena_fd`) e os decodificadores `hexa7seg` dos displays (Figura 1). Os elementos de interface externa, isto é, o detector de borda e os decodificadores, ficam somente nesse módulo. A Tabela 2 lista todos os módulos do projeto.

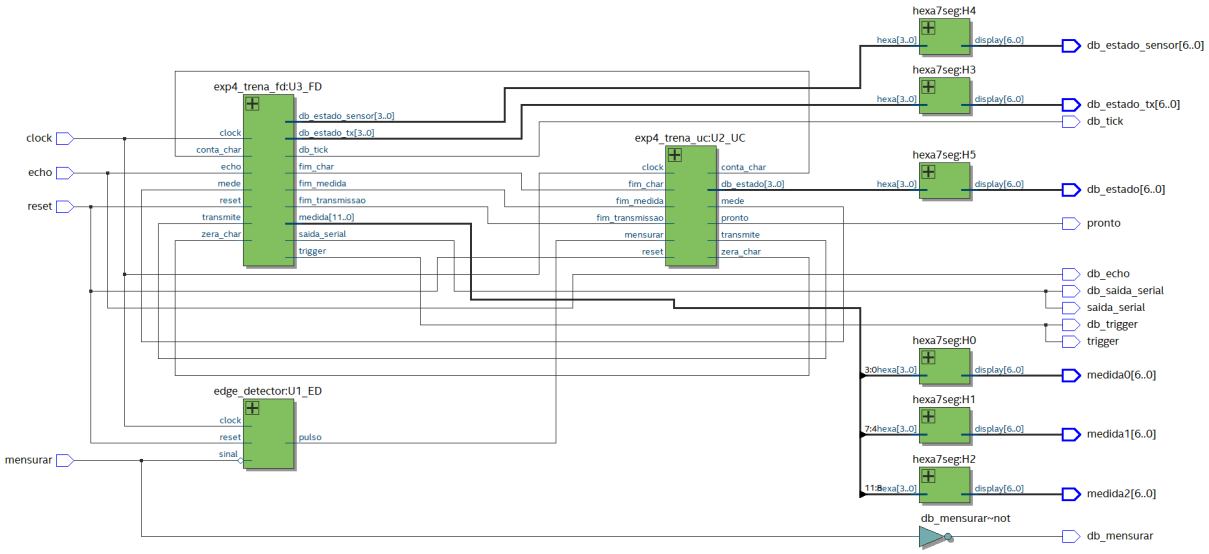


Figura 1: Diagrama RTL do módulo `exp4_trena`, obtido no RTL Viewer.

Tabela 2: Módulos do projeto.

Módulo	Função	Origem
`exp4_trena`	Módulo principal: detector de borda, UC, FD e displays	Esta experiência
`exp4_trena_uc`	UC da trena (8 estados)	Esta experiência
`exp4_trena_fd`	FD da trena: sensor, transmissor, multiplexador e contador de caracteres	Esta experiência
`interface_hcsr04` (`_uc`, `_fd`)	Dispara o sensor e mede a largura de `echo` em cm	Experiência 3 (alterado)
`tx_serial_7E1` (`_fd`, `tx_serial_uc`)	Transmissão serial 115200/7E1	Experiência 2 (refatorado)
`contador_cm` (`_fd`, `_uc`)	Conta centímetros enquanto `echo` = 1	Experiência 3
`edge_detector`	Pulso de 1 ciclo na borda de subida	Biblioteca da disciplina
`gerador_pulso`	Pulso de largura programável (`trigger`)	Biblioteca da disciplina
`contador_bcd_3digitos`	Contador BCD de 3 dígitos	Biblioteca da disciplina
`contador_m`	Contador módulo M com saídas `fim` e `meio`	Biblioteca da disciplina
`registrador_n`	Registrador de N bits	Biblioteca da disciplina
`deslocador_n`	Registrador de deslocamento de N bits	Biblioteca da disciplina
`mux_4x1_n`	Multiplexador 4x1 de N bits	Biblioteca da disciplina
`hexa7seg`	Decodificador hexadecimal para 7 segmentos	Biblioteca da disciplina

1.2.1 Fluxo de dados

O FD recebe os comandos da UC e devolve os sinais de condição `fim_medida`, `fim_transmissao` e `fim_char`. Ele é formado por quatro componentes:

- **`interface_hcsr04` (U1_SENSOR):** ao receber `mede`, gera o `trigger`, mede a largura de `echo` e guarda a distância (3 dígitos BCD, 12 bits) em um registrador. Seu `pronto` é o `fim_medida`.
- **`tx_serial_7E1` (U2_TX):** ao receber `transmite`, envia o caractere presente em sua entrada `dados_ascii`. Seu `pronto` é o `fim_transmissao`.
- **`mux_4x1_n` de 7 bits (U3_MUX):** escolhe o caractere a enviar. As entradas são D0 = centena, D1 = dezena, D2 = unidade e D3 = 23H (`#`). O código ASCII de cada dígito é obtido concatenando `011` ao dígito BCD (`{3'b011, bcd}` = 30H + dígito).

- **`contador_m` com $M = 4$ (U4_CHAR):** gera o seletor do multiplexador. `zera_char` seleciona a centena, `conta_char` avança para o caractere seguinte e `fim_char` fica ativo quando o seletor aponta para o `#`.

Tabela 3: Sinais de controle (UC → FD) e de condição (FD → UC).

Sinal	Sentido	Efeito
`mensurar`	Detector → UC	Pulso de 1 ciclo na borda de subida do botão.
`zera_char`	UC → FD	Zera o contador de caracteres (seleciona a centena).
`mede`	UC → FD	Pulso de 1 ciclo que inicia uma medida no sensor.
`transmite`	UC → FD	Pulso de 1 ciclo que inicia o envio de um caractere.
`conta_char`	UC → FD	Avança o contador de caracteres.
`fim_medida`	FD → UC	A medida terminou e está no registrador.
`fim_transmissao`	FD → UC	O quadro serial do caractere atual terminou.
`fim_char`	FD → UC	O caractere selecionado é o último (`#`).

1.2.2 Unidade de controle

A UC é uma máquina de Moore com 8 estados, descrita pelo diagrama ASM da Figura 2 e pela Tabela 4. Os números entre parênteses são os códigos mostrados em `db_estado` (HEX5).

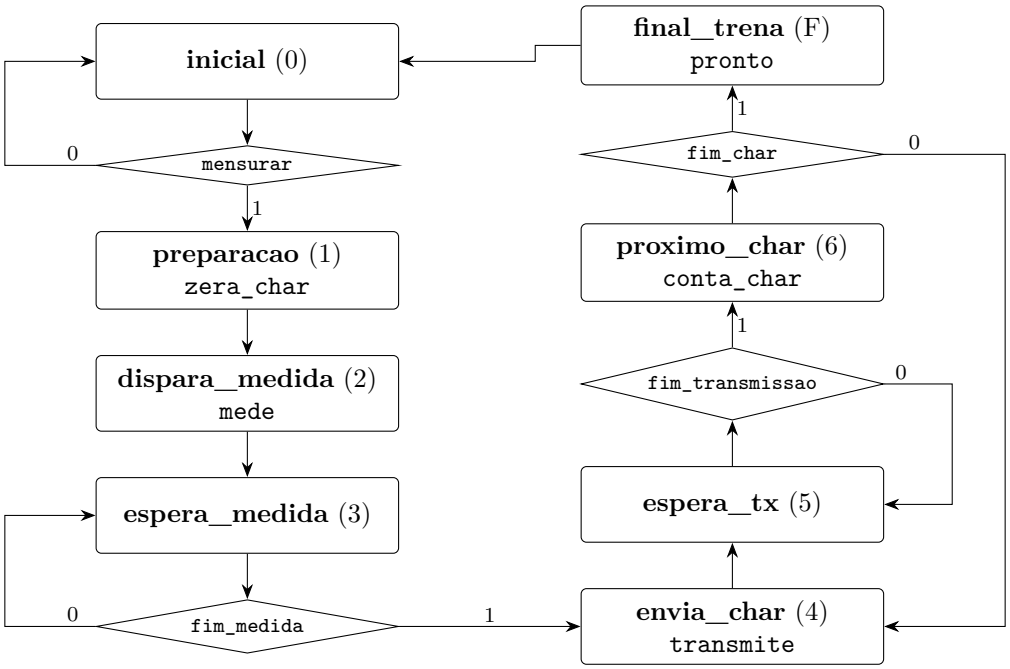


Figura 2: Diagrama ASM da UC da trena (`exp4_trena_uc`), com as saídas de Moore dentro dos estados. Qualquer código de estado não utilizado leva ao estado `inicial`.

Tabela 4: Estados, saídas e transições da UC da trena.

Estado (`db_estado`)	Saída ativa	Condição	Próximo estado
`inicial` (0)	—	`mensurar` = 1	`preparacao`
		`mensurar` = 0	`inicial`
`preparacao` (1)	`zera_char`	—	`dispara_medida`
`dispara_medida` (2)	`mede`	—	`espera_medida`
`espera_medida` (3)	—	`fim_medida` = 1	`envia_char`
		`fim_medida` = 0	`espera_medida`
`envia_char` (4)	`transmite`	—	`espera_tx`
`espera_tx` (5)	—	`fim_transmissao` = 1	`proximo_char`
		`fim_transmissao` = 0	`espera_tx`
`proximo_char` (6)	`conta_char`	`fim_char` = 1	`final_trena`
		`fim_char` = 0	`envia_char`
`final_trena` (F)	`pronto`	—	`inicial`

Em `proximo_char`, o seletor ainda aponta para o caractere que acabou de ser enviado. Por isso `fim_char` = 1 nesse estado significa que o `#` já foi transmitido e a operação terminou.

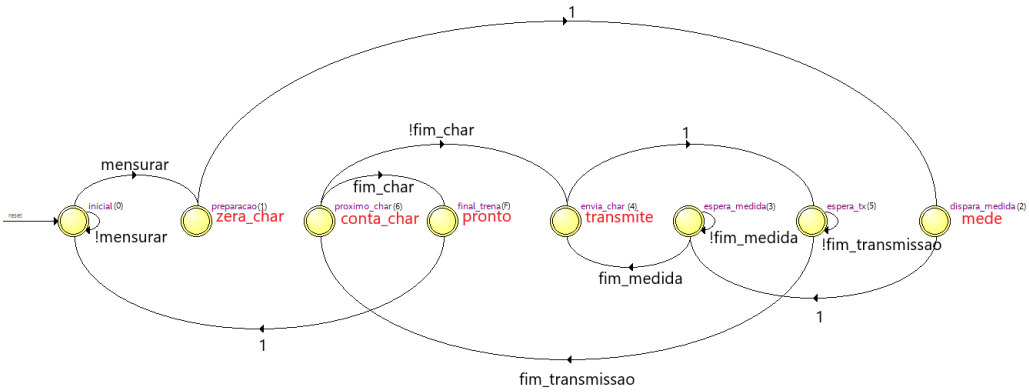


Figura 3: Diagrama de estados da UC da trena obtido no State Machine Viewer, anotado com as condições de transição (em preto) e os sinais de controle (em vermelho).

1.3 Funcionamento

Uma operação completa, para um objeto a 172 cm, segue os passos abaixo. Os números entre parênteses são os códigos de estado das UCs, e os tempos estão na Tabela 5.

1. **Repouso.** As três UCs estão em `inicial` (0), `saida_serial` está em '1' e os displays mostram a última medida.

2. **Acionamento.** Ao pressionar `KEY0`, o detector de borda gera um pulso de 1 ciclo e a UC da trena passa por `preparacao` (1), que zera o contador de caracteres, e por `dispara_medida` (2), que envia `mede` ao sensor. A UC da trena então aguarda em `espera_medida` (3).

3. **Disparo do sensor.** A UC do sensor zera o contador de centímetros e o registrador da medida, e por isso os displays passam a mostrar 000. Em seguida, gera o `trigger` de 10 μs e aguarda a subida de `echo` em `espera_echo` (3).

4. **Medida.** Enquanto `echo` = 1, a UC do sensor fica em `medida` (4) e o `contador_cm` soma 1 cm a cada 2941 ciclos de clock. Para 172 cm, o `echo` dura cerca de 10,1 ms.

5. **Registro.** Na descida de `echo`, o valor 172 é carregado no registrador (`armazenamento`, 5) e passa a aparecer nos displays. O estado `final_medida` (F) ativa `fim_medida`.

6. **Envio.** Para cada um dos quatro caracteres (`1`, `7`, `2` e `#`), a UC da trena percorre `envia_char` (4), `espera_tx` (5) e `proximo_char` (6). O transmissor monta o quadro 7E1 do caractere selecionado e o envia em cerca de 95,5 μs.

7. **Fim.** Depois do `#`, a UC vai para `final_trena` (F), ativa `pronto` por 1 ciclo e volta a `inicial`. Os displays mantêm 172 até o próximo acionamento ou até o reset.

Tabela 5: Temporização do circuito (clock de 50 MHz, período de 20 ns).

Grandeza	Ciclos de clock	Tempo
Largura do `trigger`	500	10 μs
Largura de `echo` por centímetro	2941	$\approx$ 58,8 μs
`echo` para 5 cm	14 705	$\approx$ 294 μs
`echo` para 35 cm	102 935	$\approx$ 2,06 ms
`echo` para 100 cm	294 100	$\approx$ 5,88 ms
`echo` para 172 cm	505 852	$\approx$ 10,1 ms
Duração de um bit serial (`tick`)	434	$\approx$ 8,68 μs
Um caractere (repouso + 10 bits do quadro)	$\approx$ 4774	$\approx$ 95,5 μs
Quatro caracteres	$\approx$ 19 100	$\approx$ 382 μs

1.4 Componentes reutilizados e alterações

O roteiro pede que os circuitos das Experiências 2 e 3 sejam usados como componentes internos, sem elementos de interface externa. Esta seção descreve cada um e as alterações feitas.

1.4.1 Interface com o sensor HC-SR04

O `interface_hcsr04` tem uma UC de 7 estados (Tabela 6 e Figura 4) e um FD com três blocos: um `gerador_pulso` de 500 ciclos (10 μs) para o `trigger`, um `contador_cm` com $R = 2941$ ciclos por centímetro e um `registrador_n` de 12 bits para a medida.

O valor de R vem do tempo de ida e volta do som. Com velocidade de 340 m/s, cada centímetro de distância corresponde a $2 \cdot 1\,\text{cm}/34000\,\text{cm/s} \approx 58{,}8$ μs de `echo`, ou 58,8 μs/20 ns $\approx$ 2941 ciclos. O `contador_cm` conta o primeiro centímetro na metade desse período (saída `meio` do `contador_m`), de modo que a medida sai arredondada para o centímetro mais próximo.

Tabela 6: Estados da UC do sensor (`interface_hcsr04_uc`).

Estado (`db_estado`)	Saída	Descrição
`inicial` (0)	—	Aguarda `medir`.
`preparacao` (1)	`zera`	Zera os contadores e o registrador.
`envia_trigger` (2)	`gera`	Inicia o pulso de `trigger`.
`espera_echo` (3)	—	Aguarda a subida de `echo`.
`medida` (4)	—	Conta centímetros até a descida de `echo`.
`armazenamento` (5)	`registra`	Carrega a medida no registrador.
`final_medida` (F)	`pronto`	Sinaliza o fim da medida.

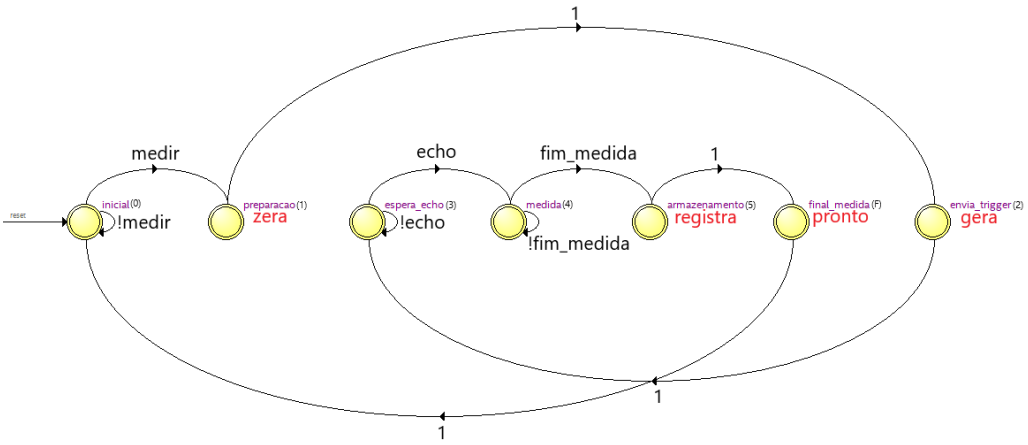


Figura 4: Diagrama de estados da UC do sensor (`interface_hcsr04_uc`) obtido no State Machine Viewer, anotado com as condições de transição (em preto) e os sinais de controle (em vermelho).

Alteração. O `interface_hcsr04` já não tinha elementos de interface externa e manteve a mesma interface. Houve uma única alteração, no alcance do `reset`. Na Experiência 3, o `reset` chegava apenas às máquinas de estados, e o fluxo de dados só era limpo pelo sinal `zera`, no início de cada nova medida. Assim, depois de um `reset` o registrador continuava com a medida anterior, o que viola R1. Agora o sinal `zera` do fluxo de dados passa por um OU com o `reset` (Listagem 1), e o `reset` zera também o gerador de pulso, o contador de centímetros e o registrador. O problema foi encontrado na simulação do circuito completo. Os módulos `interface_hcsr04_uc` e `interface_hcsr04_fd` não foram modificados.

```
     // Sinais internos
     wire        s_zera;
+    wire        s_limpa;
     wire        s_gera;

+    // O reset global tambem limpa o fluxo de dados (gerador de pulso,
+    // contador de cm e registrador da medida).
+    assign s_limpa = s_zera | reset;
+
     // Fluxo de dados
     interface_hcsr04_fd U2 (
         .clock     (clock       ),
         .pulso     (echo        ),
-        .zera      (s_zera      ),
+        .zera      (s_limpa     ),
```

Listagem 1: Alteração no `interface_hcsr04`: o `reset` passa a limpar também o fluxo de dados. Linhas com `-` foram removidas e linhas com `+` foram acrescentadas.

1.4.2 Transmissão serial 115200/7E1

O `tx_serial_7E1` usa um deslocador de 11 bits, um contador de bits módulo 12 e um gerador de tick módulo 434. Como 50 MHz/115200 ≈ 434, o erro de *baud rate* é de cerca de 0,006%. O quadro é composto por repouso (‘1’), *start bit* (‘0’), sete bits de dado com o menos significativo primeiro, bit de paridade par (redução XOR dos dados) e *stop bit* (‘1’). A Tabela 7 mostra os quadros esperados para 172 cm, e a Tabela 8 e a Figura 5 descrevem a UC.

Tabela 7: Quadros seriais esperados para 172 cm, na ordem de transmissão (da esquerda para a direita).

Caractere	ASCII	Start	d0	d1	d2	d3	d4	d5	d6	Paridade	Stop
`1`	31H	0	1	0	0	0	1	1	0	1	1
`7`	37H	0	1	1	1	0	1	1	0	1	1
`2`	32H	0	0	1	0	0	1	1	0	1	1
`#`	23H	0	1	1	0	0	0	1	0	1	1

Tabela 8: Estados da UC do transmissor (`tx_serial_uc`).

Estado (`db_estado`)	Saídas	Descrição
`inicial` (0)	—	Aguarda `partida`.
`preparacao` (1)	`zera`, `carrega`	Carrega o quadro e zera os contadores.
`espera` (3)	—	Aguarda o próximo `tick`.
`transmissao` (7)	`desloca`, `conta`	Coloca o próximo bit na saída.
`final_tx` (F)	`pronto`	Sinaliza o fim do quadro.

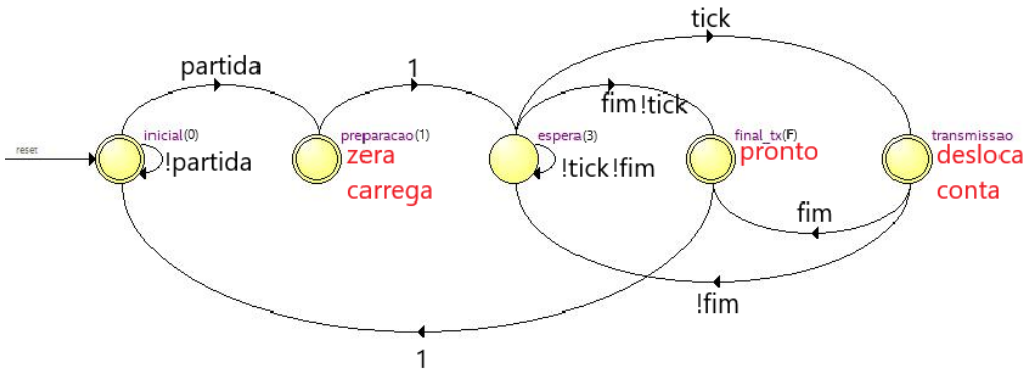


Figura 5: Diagrama de estados da UC do transmissor (`tx_serial_uc`) obtido no State Machine Viewer, anotado com as condições de transição (em preto) e os sinais de controle (em vermelho).

Refatoração. Na Experiência 2, o `tx_serial_7E1` era o circuito de topo e continha elementos de interface externa. Foram feitas três alterações (Listagens 2 e 3):

- o `edge_detector` interno foi retirado, e a entrada `partida` passou a receber diretamente o pulso de 1 ciclo gerado pela UC da trena;
- o `hexa7seg` foi retirado, e o estado da UC sai em 4 bits, decodificado somente em `exp4_trena`;
- as saídas `db_clock`, `db_partida` e `db_saida_serial`, que eram apenas cópias de outros sinais, foram retiradas, junto com os fios `s_reset`, `s_partida`, `s_partida_ed` e `s_estado`. A saída `db_tick` foi mantida.

```
 module tx_serial_7E1 (
     input        clock          ,
     input        reset          ,
-    input        partida        , // entradas
+    input        partida        , // pulso de 1 clock (vem da UC do topo)
     input [6:0]  dados_ascii    ,
     output       saida_serial   , // saidas
     output       pronto         ,
-    output       db_clock       , // saidas de depuracao
-    output       db_tick        ,
-    output       db_partida     ,
-    output       db_saida_serial,
-    output [6:0] db_estado
+    output       db_tick        , // saidas de depuracao
+    output [3:0] db_estado
 );
```

Listagem 2: Interface do `tx_serial_7E1` antes e depois da refatoração.

```
-    // detetor de borda para tratar pulsos largos
-    edge_detector U4_ED (
-        .clock ( clock        ),
-        .reset ( reset        ),
-        .sinal ( s_partida    ),
-        .pulso ( s_partida_ed )
-    );
-
     // saida serial
     assign saida_serial = s_saida_serial;

     // depuracao
-    assign db_clock        = clock;
     assign db_tick         = s_tick;
-    assign db_partida      = s_partida;
-    assign db_saida_serial = s_saida_serial;
-
-    // hexa0
-    hexa7seg HEX0 (
-        .hexa    ( s_estado  ),
-        .display ( db_estado )
-    );
```

Listagem 3: Trechos retirados do corpo do `tx_serial_7E1`: detector de borda, cópias de depuração e decodificador de estado.

1.5 Decisões de projeto e limitações

- **Serialização por multiplexador.** Os quatro caracteres são escolhidos por um multiplexador 4x1 comandado por um contador módulo 4. Os três dígitos são sempre enviados, inclusive zeros à esquerda (5 cm → `005#`).
- **Comandos por pulso de 1 ciclo.** `mede` e `transmite` são gerados em estados dedicados (`dispara_medida` e `envia_char`), que duram exatamente um ciclo. É o formato de partida que o sensor e o transmissor esperam.
- **Envio só depois da medida.** A transmissão começa após `fim_medida`, quando o registrador já contém a nova medida. Displays e porta serial leem o mesmo registrador e, portanto, mostram sempre o mesmo valor.
- **Acionamentos ignorados durante a operação.** A UC só observa o pulso do detector de borda no estado `inicial`. Manter o botão pressionado ou pressioná-lo durante uma medida não tem efeito (R2).
- **Significado de `pronto`.** `pronto` é ativado em `final_trena`, depois do envio do `#`. Nesse instante a medida já está nos displays e na porta serial, e a operação terminou.

O circuito tem as seguintes limitações:

- Não há *timeout* de `echo`. Se o sensor não responder (sensor desconectado, por exemplo), a UC do sensor permanece em `espera_echo` até o `reset`.

- `pronto` dura um ciclo de clock (20 ns) e não produz brilho perceptível em `LEDR0`. Na bancada, o fim da operação é confirmado pela chegada do `#` na porta serial.
- A faixa de apresentação é de 000 a 999 cm, enquanto o HC-SR04 mede tipicamente de 2 a 400 cm.

1.6 Plano de testes

A verificação tem duas etapas: simulação no ModelSim, com casos de teste automáticos (CT1 a CT12), e testes do circuito montado na bancada (ET1 a ET4). Os testes das etapas intermediárias da montagem (EI1 a EI3) fazem parte do plano de execução (Seção 1.8.2).

1.6.1 Procedimento de simulação

A verificação é feita de baixo para cima, do componente mais simples ao circuito completo:

1. Compilar no ModelSim todos os arquivos `.v` do projeto e os *testbenches*.
2. **Componentes.** Simular os *testbenches* de `hexa7seg`, `edge_detector`, `registrador_n`, `deslocador_n`, `mux_4x1_n`, `contador_m`, `contador_bcd_3digitos`, `gerador_pulso` e `contador_cm`.
3. **Blocos reutilizados.** Simular `interface_hcsr04_tb` e `tx_serial_7E1_tb` isoladamente, para confirmar que as alterações da Seção 1.4 não mudaram o comportamento das Experiências 2 e 3.
4. **Circuito completo.** Simular `exp4_trena_tb` com os casos CT1 a CT12 (Tabela 9).
5. Conferir o resultado de cada caso. Se algum falhar, abrir as formas de onda com os sinais da Tabela 13 para localizar o erro.

Todos os *testbenches* comparam cada resultado com o valor esperado e terminam com *todos os casos OK* ou *FALHA*. O `exp4_trena_tb` contém um modelo do HC-SR04, que responde ao `trigger` com um `echo` de largura programável cerca de 200 μs depois, e um receptor serial 115200/7E1, que decodifica `saida_serial` e confere dado, paridade e *stop bit*. Monitores conferem durante toda a simulação a largura do `trigger` (10 μs), a largura de `pronto` (1 ciclo), o período de `db_tick` (434 ciclos) e se os sinais `db_*` são cópias dos sinais correspondentes. No *testbench*, `mensurar` fica em '1' no repouso, porque o botão `KEY0` é ativo em nível baixo.

O *script* `simula_todos.do` compila as fontes, executa os doze *testbenches* na ordem acima e imprime um resumo. O `simula.do` simula um *testbench* isolado, com as formas de onda na tela.

1.6.2 Casos de teste de simulação

Tabela 9: Casos de teste de simulação do `exp4_trena_tb`.

ID	Descrição (req.)	Estímulo	Resultado esperado
CT1	Reset inicial (R1)	`reset` = 1 por alguns ciclos, sem `mensurar`.	UC em `inicial` (0), `medida` = 000, `trigger` = 0, `saida_serial` = 1 e `pronto` = 0.
CT2	Medida de 5 cm (R2, R3, R6)	Pressionar e soltar `mensurar`, com `echo` de 14 705 ciclos.	`trigger` de 10 μs, medida 005 e serial 30H, 30H, 35H, 23H.
CT3	Medida de 35 cm (R3, R6)	`echo` de 102 935 ciclos.	Medida 035 e serial 30H, 33H, 35H, 23H.
CT4	Medida de 172 cm, exemplo do roteiro (R3, R6)	`echo` de 505 852 ciclos.	Medida 172 e serial 31H, 37H, 32H, 23H, com os quadros da Tabela 7.
CT5	Transporte de dígitos e paridade zero (R3, R6)	`echo` de 294 100 ciclos (100 cm).	Medida 100 e serial 31H, 30H, 30H, 23H. O caractere `0` (30H) tem bit de paridade 0.
CT6	Botão mantido (R2)	`mensurar` ativo por mais tempo que uma medida completa.	Um único `trigger` e uma única sequência de 4 caracteres. Soltar o botão não dispara nova medida.
CT7	Acionamento durante a operação (R2)	Novo pulso de `mensurar` em `espera_medida` e, depois, em `espera_tx`.	Acionamento ignorado. Uma única sequência é concluída.
CT8	Duas medidas consecutivas (R3, R4)	Medida de 35 cm seguida de medida de 172 cm.	Displays mantêm 035 entre as medidas e passam a 172, com a serial correspondente a cada uma.
CT9	Valor mantido em repouso (R4)	Aguardar 20 ms após uma medida, sem `mensurar`.	`medida` inalterada, `saida_serial` em '1' e nenhum caractere enviado.
CT10	Sinal `pronto` (R5)	Observar `pronto` em CT2 a CT5.	Pulso de 1 ciclo em `final_trena`, depois da atualização da medida e do envio do `#`.
CT11	Reset durante a operação (R1)	`reset` = 1 em `espera_echo` e, depois, no meio de um quadro serial.	UCs em `inicial`, `saida_serial` em '1', `medida` = 000. A medida seguinte funciona normalmente.

ID	Descrição (req.)	Estímulo	Resultado esperado
CT12	Temporização serial (R6)	Monitor do intervalo entre pulsos de `db_tick` e receptor serial amostrando cada bit no centro.	Intervalo de 434 ciclos (8,68 μs), exceto um por caractere (o gerador de tick é zerado a cada quadro), e nenhum erro de paridade ou de *stop bit*.

1.6.3 Procedimento na bancada

Os testes ET1 a ET4 são feitos com o circuito completo, depois da montagem (Seção 1.8.1) e dos testes intermediários EI1 a EI3 (Seção 1.8.2):

1. Com a FPGA programada, acionar e soltar `SW0`. Conferir 000 em HEX2 a HEX0 (ET1).
2. Posicionar o objeto a 10 cm do sensor, medido com a régua, e pressionar `KEY0`. Anotar o valor dos displays e conferir os mesmos três dígitos no terminal e no decodificador UART. Repetir 5 vezes e registrar na Tabela 11 (ET2).
3. Repetir o passo anterior a 20, 50 e 100 cm (ET2).
4. Manter `KEY0` pressionado por cerca de 3 s. Conferir que o terminal e o decodificador UART recebem uma única sequência de quatro caracteres (ET3).
5. Aguardar 1 minuto sem acionar o botão e conferir que o valor nos displays não muda (ET1).
6. Fechar o terminal serial, abrir o *sketch* do Processing configurado para a porta do CP2102 e fazer uma medida (ET4).

Tabela 10: Casos de teste na bancada.

ID	Descrição (req.)	Resultado esperado
ET1	Reset e valor mantido (R1, R4)	Displays em 000 após o reset. Após uma medida, o valor não muda sem novo acionamento.
ET2	Medida real (R3, R6)	Displays e terminal mostram o mesmo valor, próximo da distância medida com a régua.
ET3	Botão mantido (R2)	Uma única medida e um único envio serial.
ET4	Aplicação gráfica	A distância na tela do Processing coincide com os displays.

A diferença em relação à régua vem da resolução de 1 cm do circuito, da incerteza no posicionamento do objeto e da variação da velocidade do som com a temperatura ambiente.

Para cada distância, as cinco repetições fornecem a média $\overline{d}$, a exatidão E (erro da média em relação à régua) e a precisão σ (desvio padrão amostral):

$$\overline{d} = \frac{1}{N}\sum_{i=1}^{N} d_i, \qquad E = \overline{d} - d_{\text{ref}}, \qquad \sigma = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}\left(d_i - \overline{d}\right)^2}$$

Tabela 11: Coleta de medidas de ET2.

Repetição	10 cm	20 cm	50 cm	100 cm
1	10,20 cm	20,37 cm	48,76 cm	98,27 cm
2	9,86 cm	22,76 cm	48,88 cm	99,80 cm
3	9,95 cm	19,96 cm	48,84 cm	99,83 cm
4	10,20 cm	20,49 cm	48,72 cm	99,47 cm
5	10,52 cm	20,76 cm	48,71 cm	98,61 cm
$\overline{d}$ (média)	10,15 cm	20,87 cm	48,78 cm	99,20 cm
E (exatidão)	+0,15 cm	+0,87 cm	−1,22 cm	−0,80 cm
E/d_{ref}	+1,5%	+4,3%	−2,4%	−0,8%
σ (precisão)	0,26 cm	1,10 cm	0,08 cm	0,72 cm

O erro relativo ficou abaixo de 5% em todas as distâncias. O maior desvio padrão, a 20 cm, vem da repetição 2 (22,76 cm). Sem ela, σ cai para 0,33 cm.

1.7 Resultados da simulação

A Tabela 12 mostra o que cada *testbench* verifica e o resultado da execução do `simula_todos.do`. Os doze terminaram sem divergências.

Tabela 12: Verificação funcional por simulação.

Testbench	O que verifica	Casos	Result.
Componentes			
`hexa7seg_tb`	Os 16 códigos de 7 segmentos, montados a partir da lista de segmentos acesos de cada dígito.	16	OK
`edge_detector_tb`	Pulso de 1 ciclo na borda de subida. Sinal mantido em '1', borda de descida e `reset` não geram pulso.	6	OK
`registrador_n_tb`	Carga com `enable`, retenção com `enable` = 0 e prioridade do *clear* assíncrono.	5	OK

Testbench	O que verifica	Casos	Result.
`deslocador_n_tb`	Carga do quadro de 11 bits, deslocamento à direita com entrada serial em '1' e retenção.	5	OK
`mux_4x1_n_tb`	Seleção das quatro entradas pelo `SEL`.	10	OK
`contador_m_tb`	Contagem módulo $M = 2941$, *clear* síncrono e assíncrono, saídas `fim` ($Q = M - 1$) e `meio` ($Q = M/2 - 1$).	7	OK
`contador_bcd_3digitos_tb`	Transporte entre os dígitos (009→010, 099→100), `fim` em 999 e volta a 000.	6	OK
`gerador_pulso_tb`	Pulso de 500 ciclos (10 μs), `pronto` de 1 ciclo, interrupção por `para` e `reset` durante o pulso.	5	OK
`contador_cm_tb`	Conversão de largura de pulso em centímetros com $R = 2941$: truncamento, arredondamento de meio centímetro e limite de 999 cm.	7	OK
Blocos reutilizados			
`interface_hcsr04_tb`	Sequência de estados do sensor, `trigger` de 10 μs, medida em BCD e `reset` durante a medida (que agora zera a medida).	7	OK
`tx_serial_7E1_tb`	Quadro 7E1 com partida por pulso de 1 ciclo, paridade par em '0' e em '1', caracteres consecutivos e `reset` no meio do quadro.	8	OK
Circuito completo			
`exp4_trena_tb`	Casos CT1 a CT12 da Tabela 9.	12	OK

As Figuras 6 a 14 mostram as formas de onda no ModelSim. Além das entradas e saídas do módulo, as janelas trazem os sinais internos da Tabela 13, com os estados das três UCs apresentados pelo nome. As marcações em vermelho foram acrescentadas às capturas para destacar o que cada figura comprova.

Tabela 13: Sinais internos acompanhados nas formas de onda do `exp4_trena_tb` (*dut* = instância de `exp4_trena`).

Sinal (hierarquia a partir de *dut*)	Finalidade
`U2_UC.Eatual`	Estado da UC da trena.
`U3_FD.U1_SENSOR.U1.Eatual`	Estado da UC do sensor.
`U3_FD.U2_TX.U2_UC.Eatual`	Estado da UC do transmissor.
`s_mede`, `s_transmite`, `s_conta_char`, `s_zera_char`	Comandos da UC ao FD.
`s_fim_medida`, `s_fim_transmissao`, `s_fim_char`	Condições do FD para a UC.
`s_mensurar`, `s_mensurar_ed`	Botão já invertido e pulso do detector de borda.
`s_medida`	Medida BCD de 12 bits registrada.
`U3_FD.s_dados_ascii`, `U3_FD.s_sel`	Caractere ASCII selecionado e seletor do multiplexador.
`db_tick`	Base de tempo do *baud rate*.

1.7.1 Circuito completo

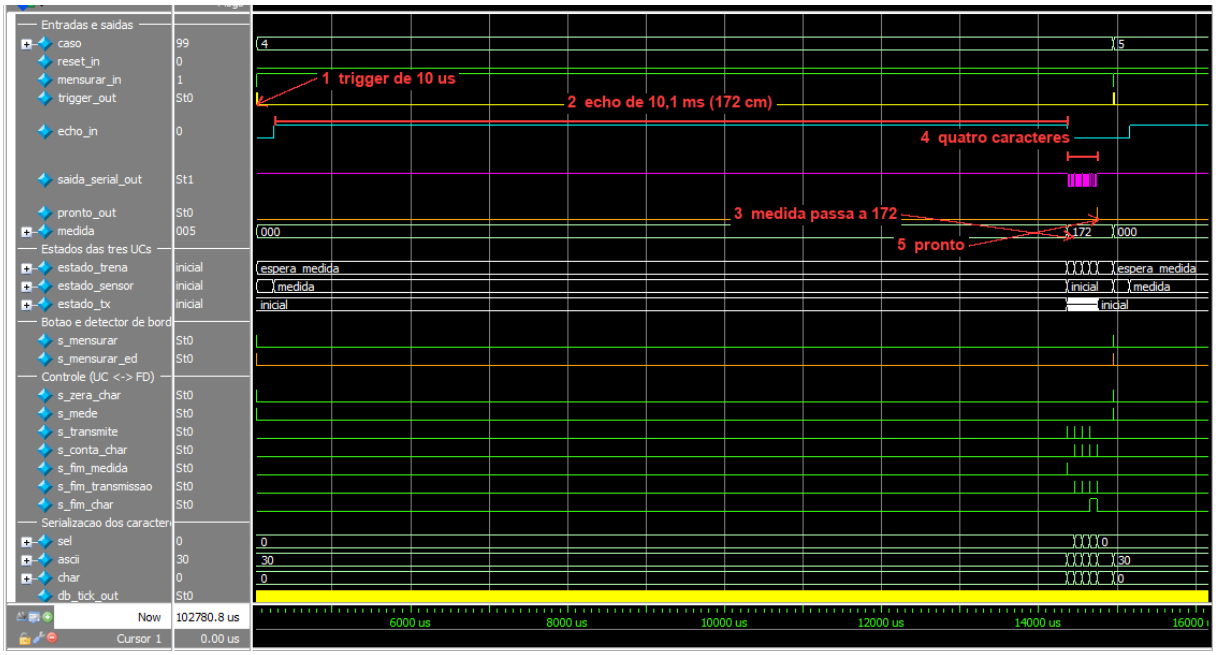


Figura 6: Visão geral de uma medida de 172 cm (CT4): sequência de estados das três UCs, `trigger`, `echo` de 10,1 ms, `medida` passando de 000 para 172 antes do envio e pulso de `pronto` no fim. Os pulsos em `saida_serial_out` à direita são os quatro caracteres.

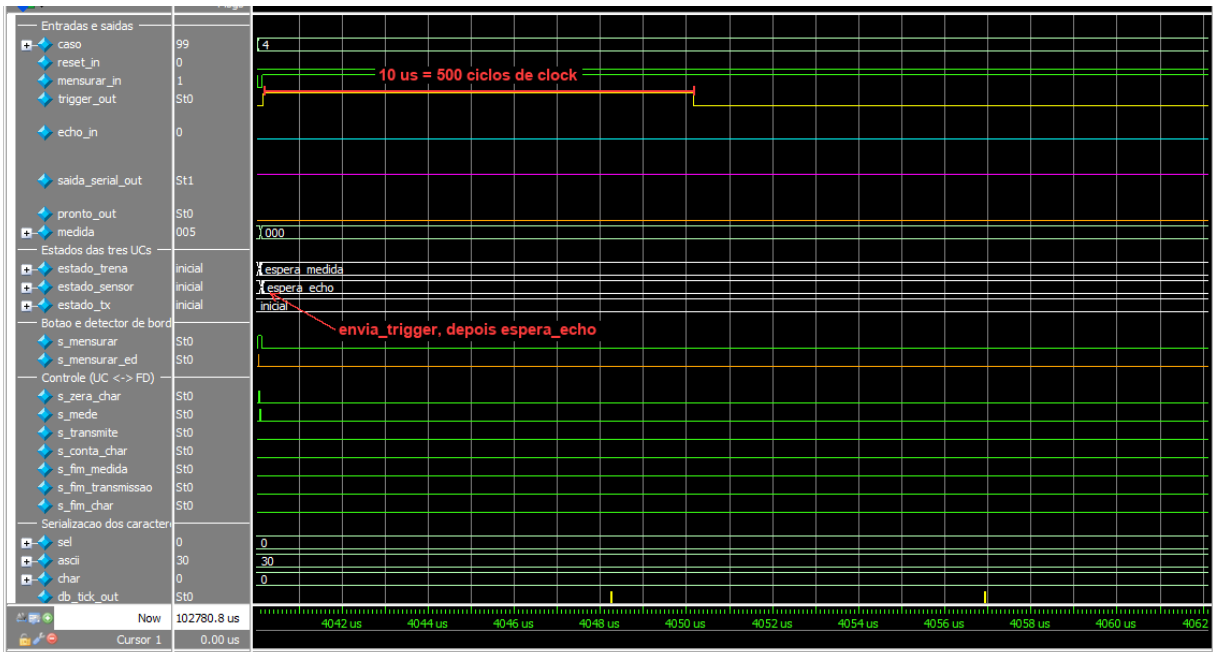


Figura 7: Detalhe do `trigger`: pulso de 10 μs (500 ciclos) disparado por `s_mede`, com `estado_sensor` passando por `envia_trigger` e parando em `espera_echo`.

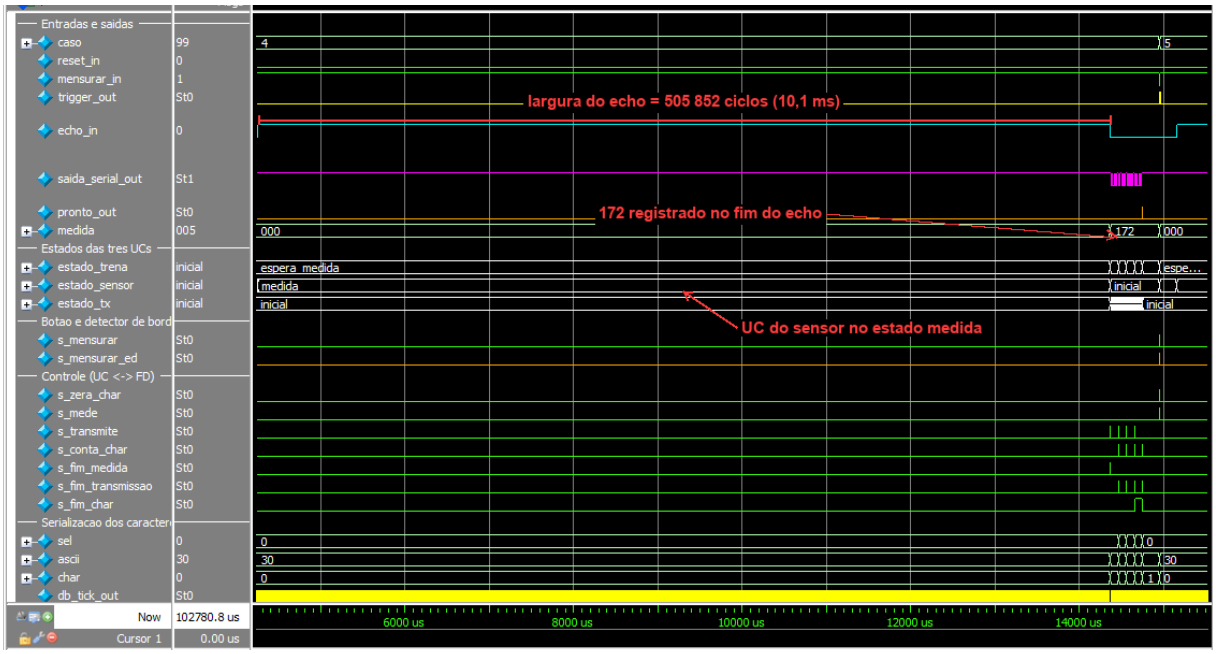


Figura 8: Detalhe da medição: `echo` de 505 852 ciclos (10,1 ms) com `estado_sensor` em `medida`, e o valor 172 registrado em `medida` quando o `echo` termina.

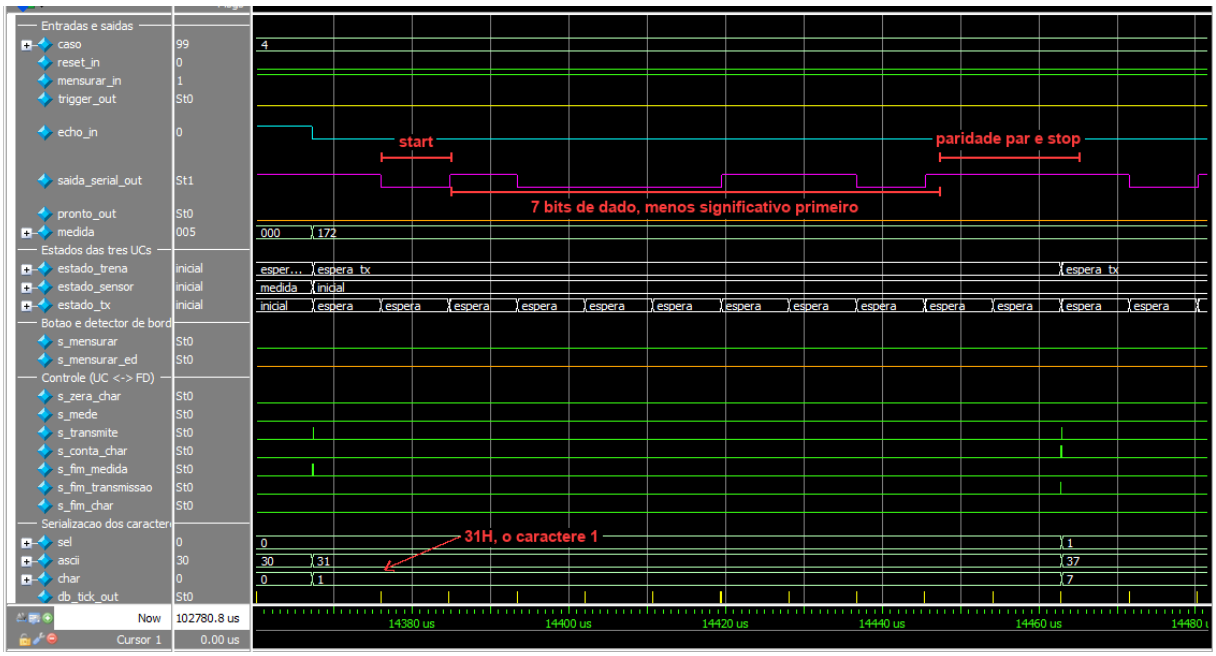


Figura 9: Envio do primeiro caractere (1, 31H): `db_tick_out` marca cada bit, `estado_tx` alterna entre `espera` e `transmissao`, e `saida_serial_out` mostra o quadro com *start bit*, sete bits de dado, paridade par e *stop bit*.

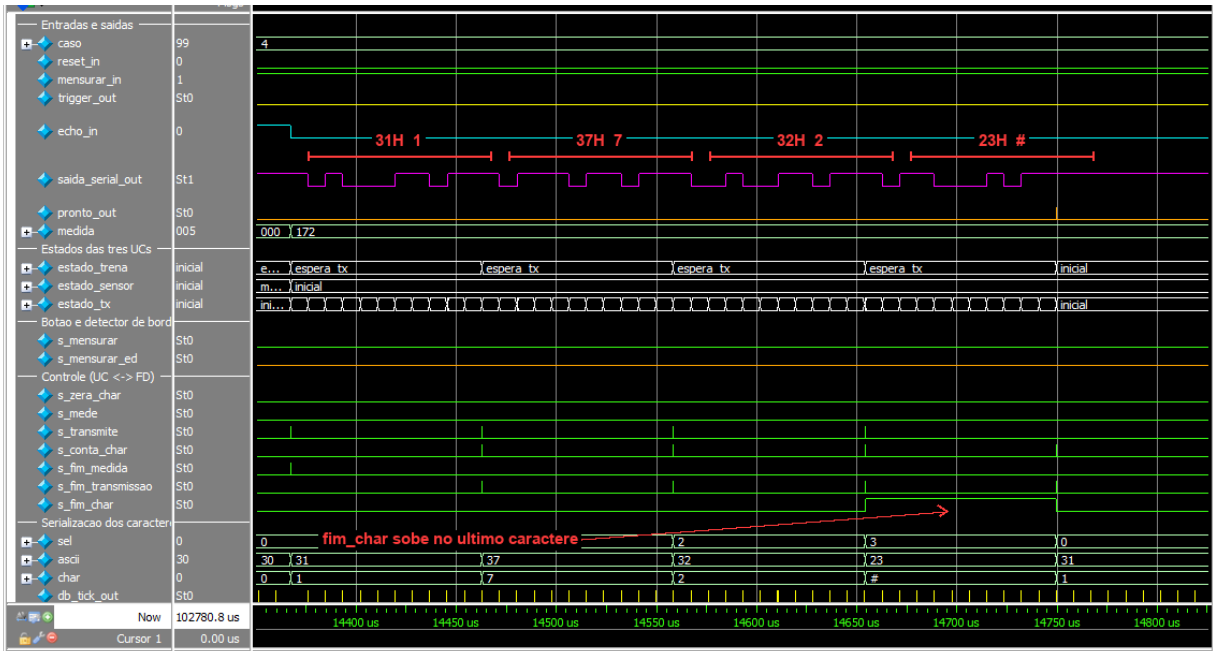


Figura 10: Sequência dos quatro caracteres: `sel` vai de 0 a 3, `ascii` passa por 31H, 37H, 32H e 23H e `s_fim_char` sobe no último, encerrando a operação.

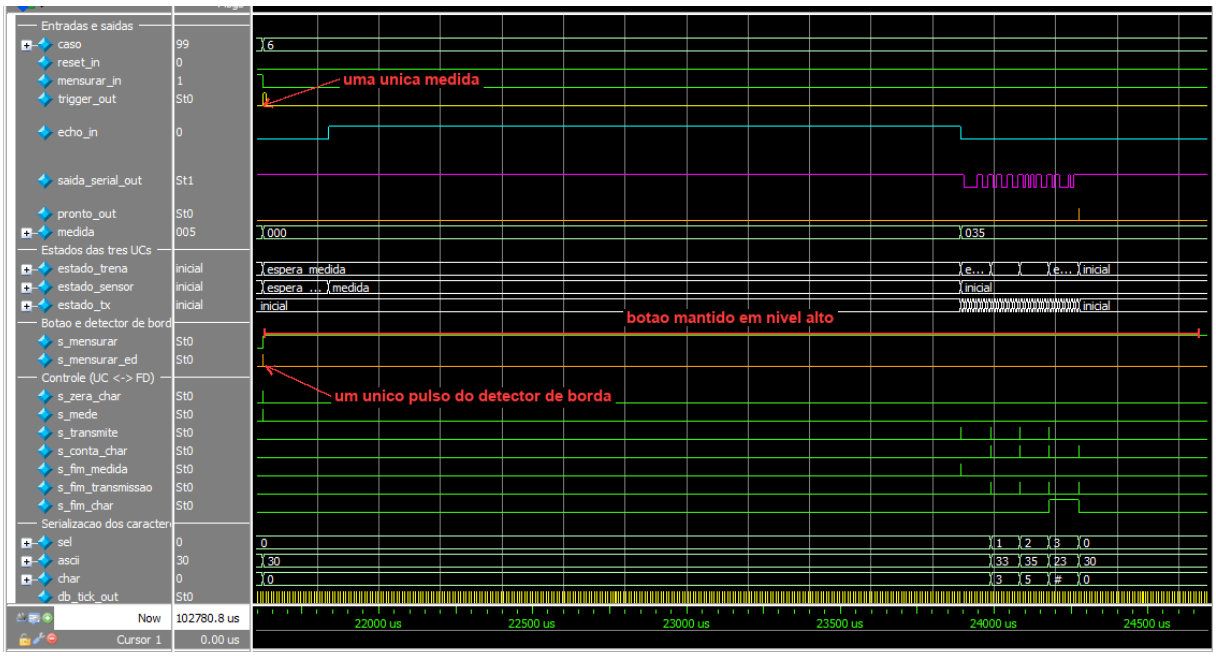


Figura 11: Botão mantido pressionado (CT6): `s_mensurar` fica em '1' durante toda a medida, mas o detector de borda gera um único pulso em `s_mensurar_ed` e só uma medida acontece.

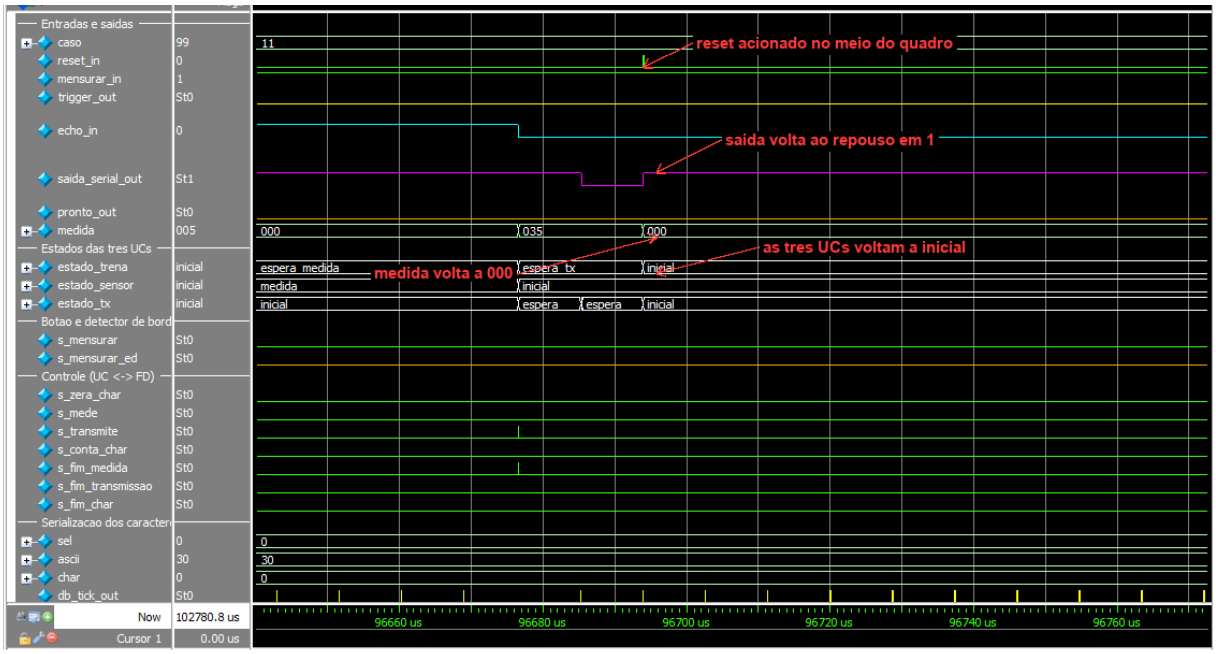


Figura 12: Reset no meio do quadro serial (CT11): as três UCs voltam a `inicial`, `medida` volta a 000 e `saida_serial_out` volta ao repouso em '1'.

1.7.2 Componentes reutilizados

As duas figuras a seguir mostram os blocos das Experiências 2 e 3 simulados fora do `exp4_trena`.

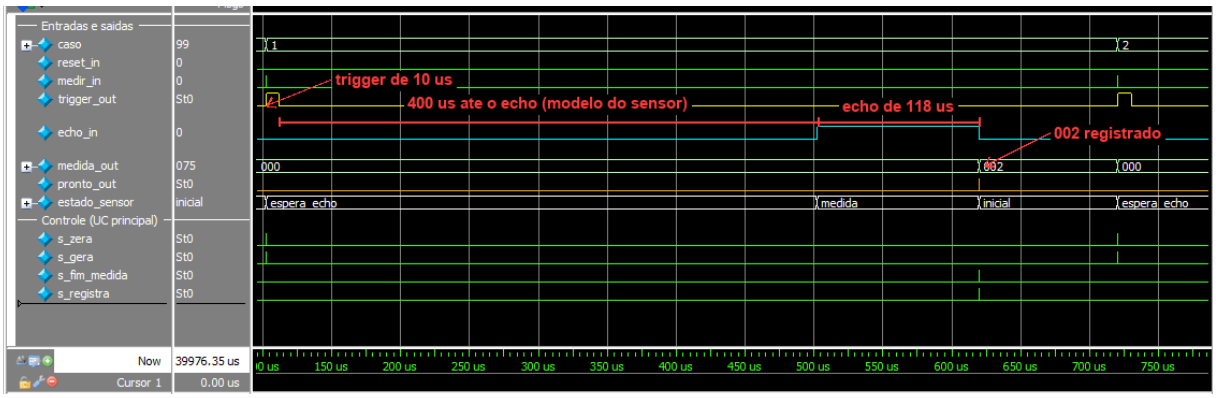


Figura 13: `interface_hcsr04` isolado (caso 1 do *testbench*): `trigger` de 10 μs, 400 μs até o `echo` no modelo do sensor desse *testbench*, sequência de estados da Tabela 6, `medida_out` = 002 registrada por `s_registra` e pulso de `pronto`.

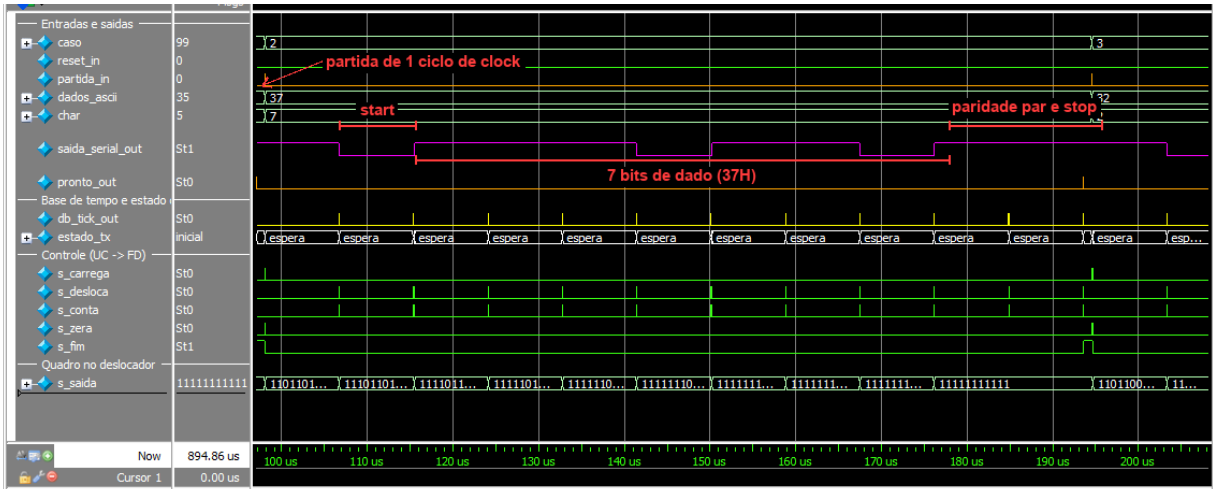


Figura 14: `tx_serial_7E1` isolado (caractere `7`, 37H): `partida` de 1 ciclo, quadro em `saida_serial_out` conforme a Tabela 7 e deslocador `s_saida` esvaziando um bit a cada `s_desloca`, até `s_fim` e `pronto`.

1.8 Plano de execução experimental

1.8.1 Plano de montagem

1. Com a placa desligada, ligar o GND da DE0-CV ao GND do HC-SR04, do módulo CP2102 e do Analog Discovery (entradas 1− e 2− do Scope). Todos precisam da mesma referência.

2. Alimentar o HC-SR04 pelo pino 29 (3,3 V) do conector GPIO_1, e não pelo pino 11 (5 V): os pinos de E/S da FPGA trabalham com 3,3 V. Os pinos 12 e 30 de cada conector são GND.

3. Ligar os sinais conforme a Tabela 14.

4. Ligar o CP2102 ao computador pelo cabo micro USB e identificar a porta serial no sistema operacional.

5. Ligar o Analog Discovery ao computador e abrir o WaveForms.
6. Programar a FPGA, acionar `SW0` e conferir 000 em HEX2 a HEX0.
7. Configurar o terminal serial em 115200/7E1 (115200 bauds, 7 bits de dados, paridade par, 1 *stop bit*).
8. Configurar o *sketch* do Processing com o nome da porta serial do CP2102 (variável `porta`). O formato 115200/7E1 já está definido no *sketch*. O terminal e o Processing não podem usar a porta ao mesmo tempo.

Tabela 14: Ligações da montagem experimental, com a numeração dos conectores de 40 pinos.

Origem (DE0-CV)	Pino	Destino	Sinal
`GPIO_1_D1`	2	Pino TRIG do HC-SR04	`trigger`
`GPIO_1_D3`	4	Pino ECHO do HC-SR04	`echo`
`GPIO_1_D33`	38	CH1+ do Scope	`db_trigger`
`GPIO_1_D35`	40	CH2+ do Scope	`db_echo`
`GPIO_1` 3,3 V	29	VCC do HC-SR04	alimentação
`GPIO_1` GND	30	GND do HC-SR04	referência
`GPIO_0_D1`	2	Pino RXD do módulo CP2102	`saida_serial`
`GPIO_0_D35`	40	DIO7 do Analog Discovery (Protocol)	`db_saida_serial`
`GPIO_0` GND	12, 30	GND do CP2102 e do Analog Discovery (Scope 1− e 2−)	referência

1.8.2 Testes das etapas intermediárias

Cada teste isola uma parte da montagem, para que uma falha no teste do circuito completo possa ser atribuída a uma ligação específica.

EI1 – Porta serial do computador. Com o CP2102 ligado apenas ao computador, unir os pinos TXD e RXD do módulo (*loopback*) e digitar caracteres no terminal configurado em 115200/7E1. *Esperado:* os caracteres digitados retornam ao terminal. *Se falhar:* conferir a porta selecionada e o driver do CP2102.

EI2 – Sinais do sensor. Com o sensor ligado à FPGA e um objeto à frente, pressionar `KEY0` e observar `db_trigger` e `db_echo` no Scope (configuração na Tabela 15). *Esperado:* `trigger` de 10 μs e, algumas centenas de microssegundos depois, `echo` com largura de cerca de 58,8 μs por centímetro. *Se falhar:* sem `trigger`, conferir a programação da FPGA e `SW0` em '0'. Com `trigger` e sem `echo`, conferir alimentação, GND e o fio ECHO.

EI3 – FPGA e porta serial, sem sensor. Desconectar o sensor e ligar a saída W1 do Wavegen ao pino de `echo`, com onda quadrada de 0 a 3,3 V, largura de 1,0 ms e período de 50 ms. Pressionar `KEY0`. *Esperado:* displays, terminal e decodificador UART mostram 017 (1,0 ms / 58,8 μs $\approx$ 17 cm), ou seja, 30H, 31H, 37H e 23H,

sem erro de paridade. Se o botão for pressionado durante um pulso (2% de chance), a leitura sai menor e deve ser repetida. *Se falhar:* com o decodificador correto e o terminal errado, conferir o fio `GPIO_0_D1` → RXD e o GND do CP2102.

1.8.3 Plano de depuração e monitoramento

A Tabela 15 mostra como cada sinal é acompanhado na bancada. O Scope usa uma única base de tempo por aquisição, por isso `trigger` e `echo` são medidos em duas aquisições, ambas disparadas pela borda de subida de CH1.

Tabela 15: Monitoramento dos sinais na bancada.

Sinal	Ferramenta	Configuração	Resultado esperado
`db_trigger`	Scope CH1	Aquisição 1: gatilho na subida de CH1, modo *Single*, 5 μs/div, 1 V/div.	Pulso de 10 μs, de 0 a 3,3 V.
`db_echo`	Scope CH2	Aquisição 2: mesmo gatilho, 2 ms/div.	Pulso de largura $\approx$ 58,8 μs $\times$ distância em cm (5,9 ms a 100 cm).
`db_saida_serial`	Protocol (UART), DIO7	115200 bauds, 7 bits, paridade par, 1 *stop bit*, repouso em '1'.	Três dígitos e `#`, sem erro de paridade ou de quadro, em cerca de 382 μs.
`db_mensurar`	LEDR1	—	Aceso enquanto `KEY0` está pressionado.
`pronto`	LEDR0	—	Sem brilho perceptível (pulso de 20 ns, Seção 1.5).
`db_estado`	HEX5	—	0 em repouso.
`db_estado_sensor`	HEX4	—	0 em repouso.
`db_estado_tx`	HEX3	—	0 em repouso.
`db_tick`	LEDR2	—	Praticamente apagado: pulso de 20 ns a cada 8,68 μs, também em repouso.

Uma operação completa dura no máximo algumas dezenas de milissegundos, então os displays HEX3 a HEX5 voltam a 0 antes que os estados intermediários sejam percebidos. Eles servem para identificar onde o circuito parou quando algo falha, conforme a Tabela 16.

Tabela 16: Diagnóstico de falhas na bancada.

Sintoma	Causa provável	Ação
Nada acontece ao pressionar `KEY0`, com LEDR1 respondendo ao botão.	`SW0` em '1' mantém o circuito em reset.	Baixar `SW0`.
HEX5 e HEX4 parados em 3.	O sensor não devolveu `echo`.	Conferir alimentação, GND e o fio ECHO, observar `db_echo` no Scope e acionar `SW0` para sair do estado.
HEX5 parado em 3 e HEX4 parado em 4.	`echo` preso em '1'.	Observar `db_echo` no Scope e conferir o fio ECHO.
Displays atualizam, o decodificador UART mostra os caracteres e o terminal não.	Falha entre a FPGA e o computador.	Conferir o fio `GPIO_0_D1` → RXD, o GND do CP2102 e a porta selecionada.
Terminal mostra caracteres errados.	Terminal ou Processing fora de 115200/7E1.	Corrigir a configuração da porta.

2 Atividade 2: Preparação do Projeto para Síntese na Placa FPGA

2.1 Designação de pinos

A Tabela 17 segue a designação mínima do roteiro. Os displays usam 7 pinos cada, mostrados no *Pin Planner* (Figura 15). Todas as portas usam o padrão 3.3-V LVTTL.

Tabela 17: Designação de pinos da Trena Digital com Saída Serial.

Sinal	Recurso da DE0-CV	Pino da FPGA	Analog Discovery
`clock`	`CLOCK_50`	M9	—
`reset`	chave `SW0`	U13	—
`mensurar`	botão `KEY0`	U7	—
`medida0`	display `HEX0`	7 pinos	—
`medida1`	display `HEX1`	7 pinos	—
`medida2`	display `HEX2`	7 pinos	—
`saida_serial`	`GPIO_0_D1`	B16	—
`trigger`	`GPIO_1_D1`	A12	—
`echo`	`GPIO_1_D3`	B12	—
`pronto`	LED `LEDR0`	AA2	—
`db_mensurar`	LED `LEDR1`	AA1	—
`db_saida_serial`	`GPIO_0_D35`	T15	Protocol – DIO7
`db_trigger`	`GPIO_1_D33`	G12	Scope – CH1+
`db_echo`	`GPIO_1_D35`	K16	Scope – CH2+
`db_estado`	display `HEX5`	7 pinos	—

2.2 Sinais de depuração adicionais

Os códigos de estado das UCs do sensor e do transmissor são convertidos por instâncias de `hexa7seg` no módulo `exp4_trena`, como o da UC da trena.

Tabela 18: Sinais de depuração acrescentados pelo grupo.

Sinal	Recurso (pino)	Utilidade
`db_estado_sensor`	HEX4 (7 pinos)	Estado da UC do sensor. Parado em 3, indica que o sensor não devolveu `echo`.
`db_estado_tx`	HEX3 (7 pinos)	Estado da UC do transmissor (0, 1, 3, 7 ou F).
`db_tick`	LEDR2 (W2)	Base de tempo do *baud rate*. É um pulso de 20 ns a cada 8,68 μs e não acende o LED de forma perceptível; é usado na simulação (CT12).

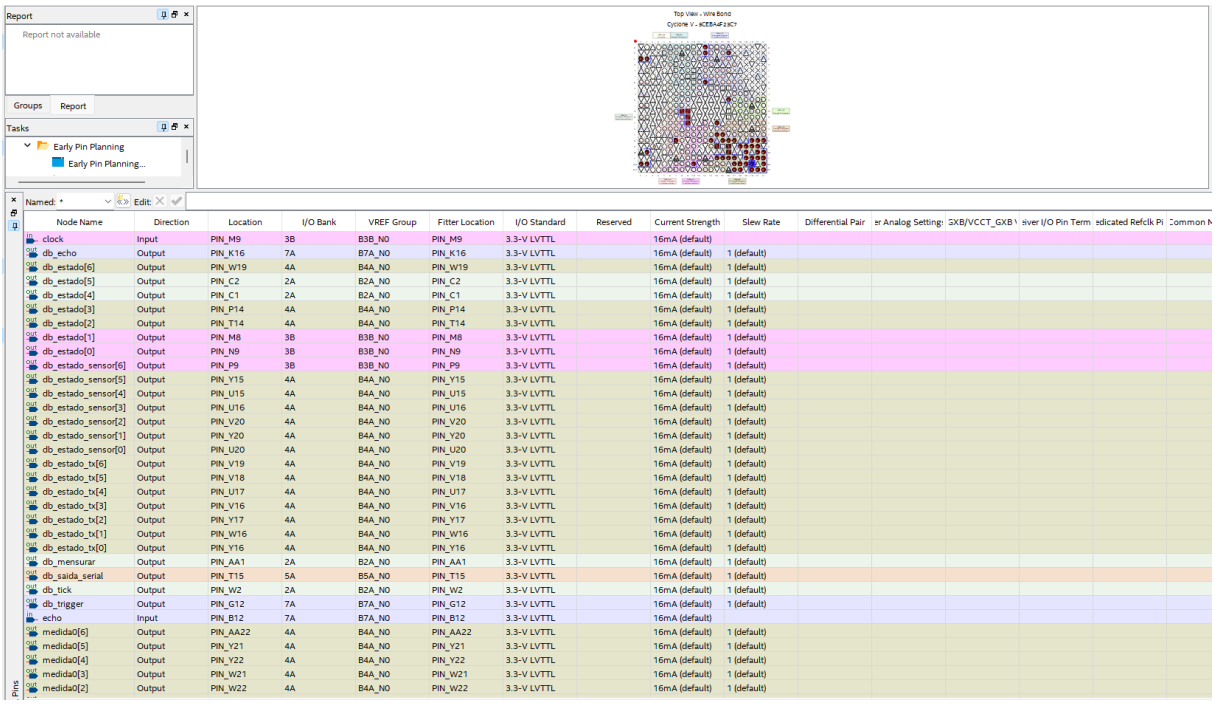


Figura 15: *Pin Planner* do Quartus Prime para o projeto `exp4_trena`: a coluna *Fitter Location* coincide com a *Location* atribuída em todas as portas.

2.3 Tratamento dos sinais ativos em nível baixo

Os botões KEY0 a KEY3 da DE0-CV são ativos em nível baixo. Como todo o projeto usa lógica ativa em alto, a entrada mensurar é invertida no módulo principal, antes do detector de borda:

```
assign s_mensurar = ~mensurar;   // KEY0 ativo em nivel baixo

edge_detector U1_ED (
    .clock (clock),  .reset (reset),
    .sinal (s_mensurar),  .pulso (s_mensurar_ed)
);
```

O db_mensurar é ligado a s_mensurar, então LEDR1 acende enquanto o botão está pressionado. Os displays de 7 segmentos também são acionados em nível baixo, mas o hexa7seg já entrega o código nesse formato. A chave SW0 (reset) é ativa em nível alto e não precisa de inversão.

2.4 Restrições de tempo e resultado da compilação

O arquivo de0_cv.sdc declara o clock de 50 MHz (create_clock com período de 20 ns) e derive_clock_uncertainty. As entradas reset e mensurar vêm de chave e botão, e echo vem do HC-SR04. Nenhuma delas tem relação de fase com o clock da placa. As saídas vão para displays, LEDs, o sensor, o CP2102 e os pinos de depuração, onde não há circuito síncrono. Por isso todas as portas recebem set_false_path, e a análise de tempo fica restrita aos caminhos entre registradores.

O projeto compila sem erros, ocupa menos de 1% do dispositivo e tem folga de tempo em todos os cantos de operação. A frequência máxima é cerca de quatro vezes a do clock da placa (Tabela 19).

Tabela 19: Resultado da compilação do projeto exp4_trena (Cyclone V 5CEBA4F23C7).

Grandeza	Valor
Lógica	115 ALMs de 18 480 (< 1%)
Registradores	151
Pinos	54 de 224 (24%)
Memória, DSP e PLL	não utilizados
Pior *slack* de *setup*	15,341 ns (canto Slow 1100 mV 0 °C)
Pior *slack* de *hold*	0,140 ns (canto Fast 1100 mV 0 °C)
F_{max}	222,92 MHz, para 50 MHz exigidos

2.5 Arquivos entregues

- Planejamento: PCS3645-T1BA5-Planejamento4.pdf;
- projeto do Quartus: exp4_T1BA5.qar;
- *testbench* do circuito: exp4_trena_tb.v;
- projeto do ModelSim: exp4_trena_ModelSim_T1BA5.zip, com os vinte arquivos Verilog do projeto, os doze *testbenches* da Tabela 12 e os *scripts* simula.do e simula_todos.do, todos na mesma pasta.

3 Atividade 4: Desafio -- Modo de Medida Dupla

3.1 Especificação

O `exp4_desafio` realiza duas medidas por acionamento de `mensurar`, exibindo e transmitindo cada uma como `ccc#`. Após enviar o primeiro `#`, espera 2 s antes de iniciar a segunda medida. Só então ativa `pronto` e volta ao repouso. Ignora novos acionamentos durante a operação; `reset` a interrompe em qualquer estado. Os requisitos R1–R8 da trena (Seção 1.1) continuam válidos. Conforme o roteiro, a mudança de comportamento fica na UC, preservando os sinais de controle dos componentes já existentes no FD; novos componentes podem ser adicionados ao FD.

3.2 Projeto

Interface e pinagem seguem as Tabelas 1 e 17. O parâmetro `CICLOS_2S` vale 100 000 000 em 50 MHz (2 s); na simulação, pode ser reduzido. O topo contém a UC, o FD, o detector de borda e os decodificadores dos displays.

3.2.1 Fluxo de dados

O FD preserva os quatro componentes da trena e acrescenta dois, destacados na Figura 16:

- `U5_TIMER_2S`: contador de 27 bits, zerado em `preparacao`, que sinaliza `fim_2s` após `CICLOS_2S` ciclos de espera.
- `U6_MEDIDAS`: registrador de 1 bit; `fim_med`=0 na primeira medida e 1 na segunda. É zerado em `preparacao` e por `reset`.

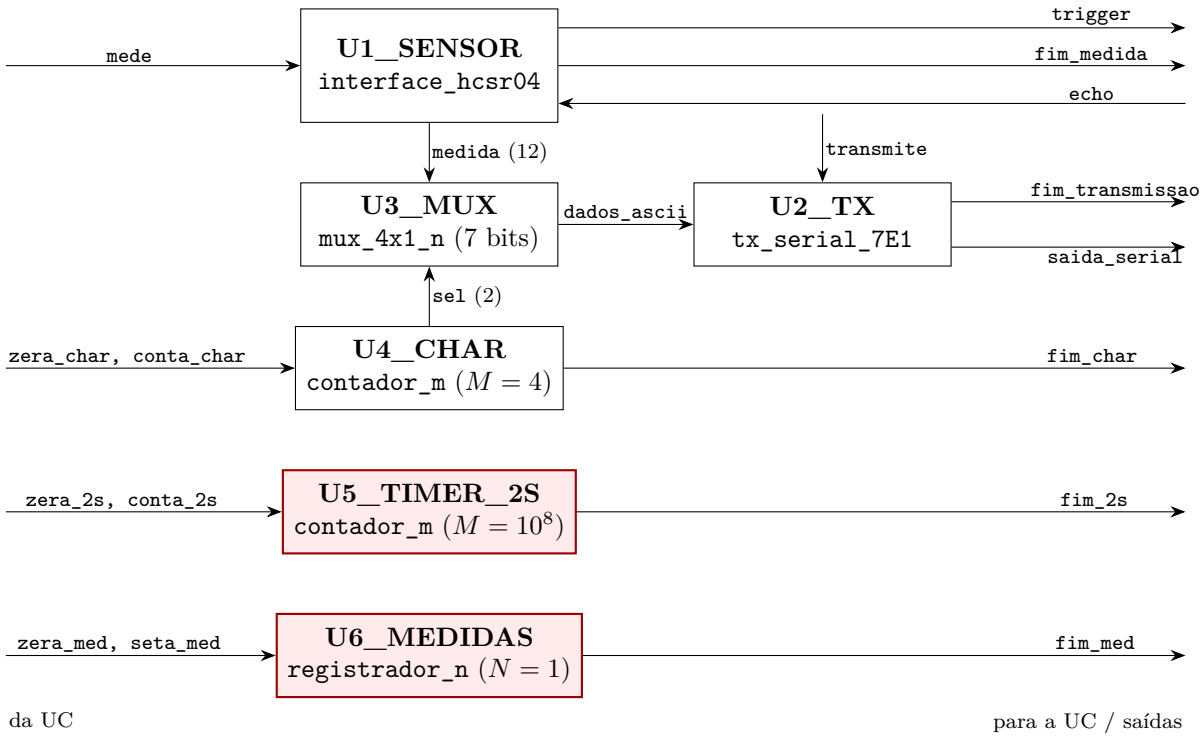


Figura 16: FD do desafio: U5 e U6 são os componentes novos.

3.2.2 Unidade de controle

A UC zera U5 e U6 em `preparacao`. Após enviar o primeiro `#`, `proximo_char` entra em `espera_2`; após o segundo, vai a `final_trena`. Em `espera_2`, a UC conta o intervalo, marca a segunda medida e então retorna a `dispara_medida`. A Figura 17 mostra os estados extraídos pelo Yosys, com suas saídas de Moore.

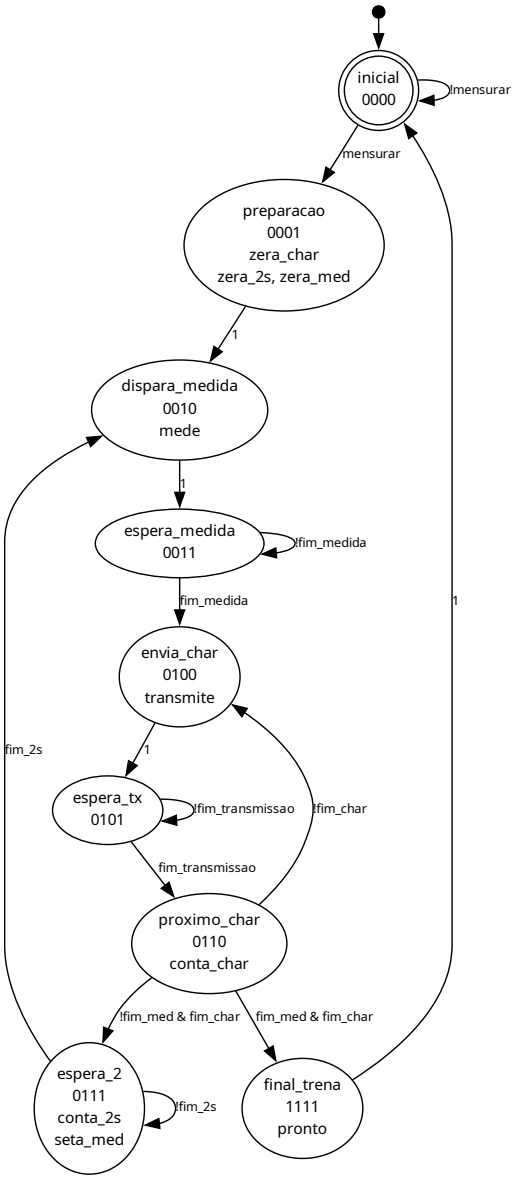


Figura 17: UC do desafio: estados, saídas ativas e transições.

3.2.3 Temporização e comportamento

O estado `espera_2` dura exatamente `CICLOS_2S` ciclos. Entre o envio do primeiro `#` e o `trigger` seguinte decorrem mais três ciclos de controle. Durante a espera, os displays

mantêm a primeira medida; no início da segunda, mostram 000 até o novo valor ser registrado. `pronto` pulsa apenas após a segunda transmissão. A UC só aceita `mensurar` em `inicial`.

3.2.4 Sinais de depuração

Não há pinos novos. Os sinais existentes expõem o estado da UC (HEX5), `trigger`, `echo` e a saída serial; a pinagem consta nas Tabelas 17 e 18.

3.3 Testes de simulação

O `exp4_desafio_tb` usa um modelo do HC-SR04 com duas distâncias e um receptor 115200/7E1. O intervalo é reduzido a 100 000 ciclos (2 ms) nos nove casos da Tabela 20. CD1–CD2 também foram executados com 100 000 000 ciclos por meio de `vsim -gCICLOS_2S=100000000 +so_cd2`.

Tabela 20: Cobertura do `exp4_desafio_tb`.

Casos	Verificação
CD1–CD2	Reset; duas medidas de 35 cm, `035#035#`, dois `trigger`, um `pronto` e intervalo em ciclos.
CD3–CD5	35/172 cm, 100/5 cm e repouso sem terceira medida.
CD6–CD7	Acionamentos extras durante a operação e botão mantido: apenas duas medidas.
CD8–CD9	Reset no intervalo e nova operação; serial 7E1 sem erros de paridade ou *stop bit*.

3.4 Resultados da simulação

No ModelSim, os nove casos terminaram com resultado OK. A UC ficou 100 000 ciclos em `espera_2` na simulação abreviada e 100 000 000 ciclos (2 s) em CD1–CD2 com o parâmetro real. As Figuras 18, 19 e 20 mostram, respectivamente, medidas diferentes, acionamentos ignorados e reset durante a espera. Nas capturas, 2 ms representam o intervalo nominal de 2 s.

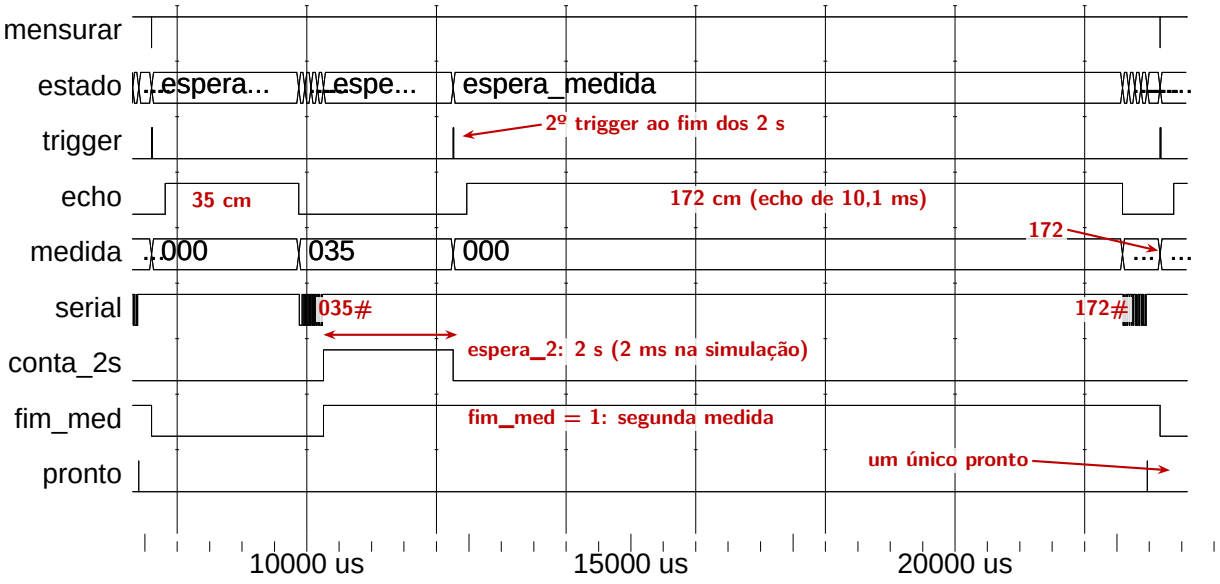


Figura 18: CD3: `035#172#`, com `pronto` após o segundo envio.

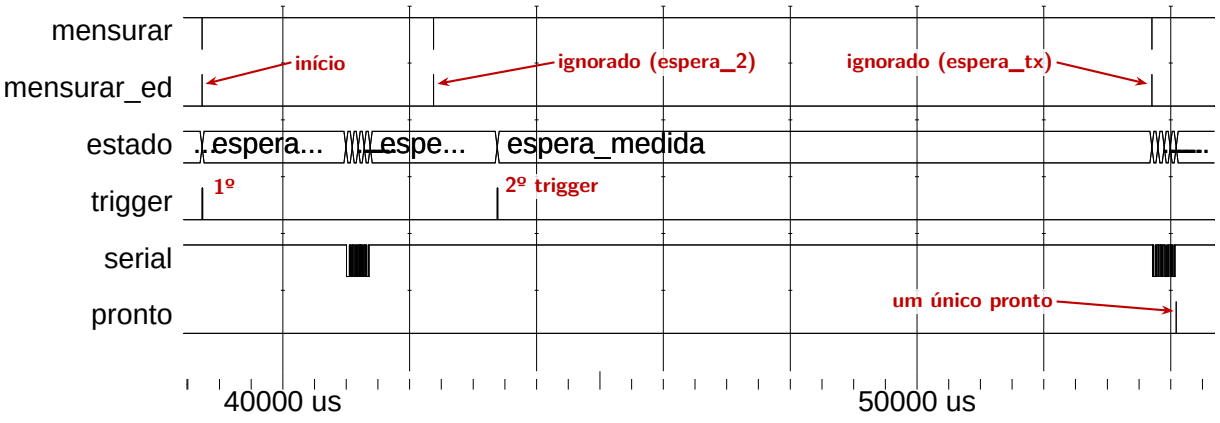


Figura 19: CD6: acionamentos extras não iniciam outra operação.

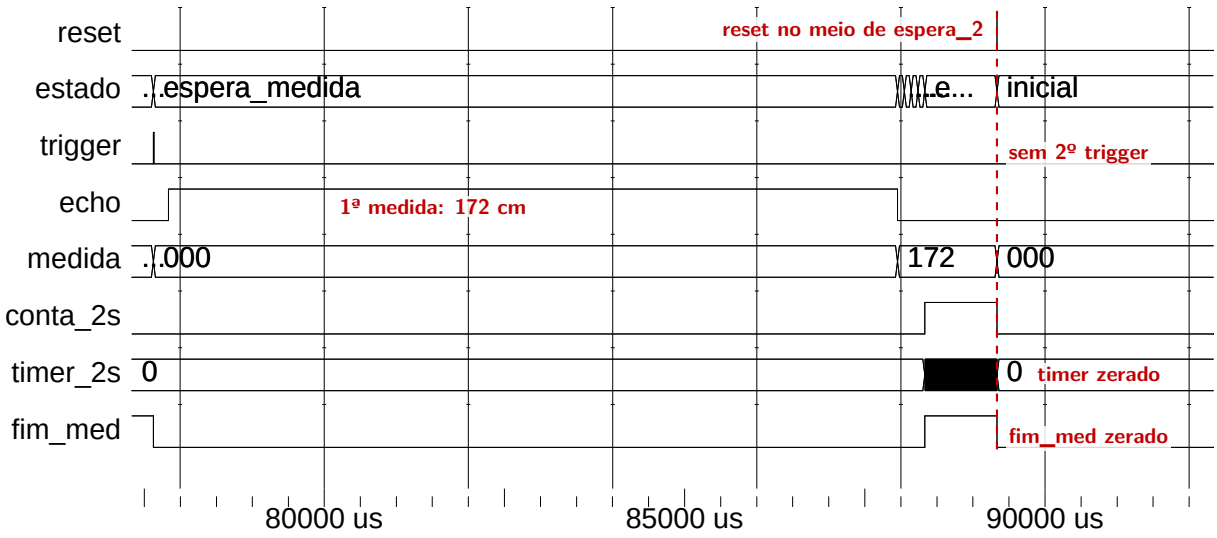


Figura 20: CD8: reset durante a espera cancela a segunda medida.

3.5 Validação

O funcionamento do desafio foi validado e apresentado ao professor.